Contents

Connect Four — Implementation Plan

> **Status:** Draft, 2026-05-17. Supersedes `Platform_Implementation_Plan.md` §Phase 4 — the original "drop in another game package" framing pre-dated the 2026 design lock that elevated Connect 4 into a multi-phase architectural initiative.
>
> **Related docs:** - `Game_SDK_Developer_Guide.md` — contract every game package must conform to - `Platform_Implementation_Plan.md` — broader platform roadmap; Phase 3.8 (multi-skill bots) is the prerequisite and is shipped - `Help_Corpus/matches-and-games.md`, `match-formats.md`, `match-design-rationale.md`, `solved-games-and-master.md` — match semantics + Master tier (already published to corpus) - `Help_Corpus/algorithm-history-*.md`, `the-history-of-game-ai.md` — algorithm background pack (already published to corpus)

Status at a glance (2026-05-17)

Sprint	Status	Items	Notes
Phase A — Architectural alignment (TTT only)			
A1 — SDK + game-as-prefix routing + slug rename	shipped	10/10	Legacy `xo` alias kept active via `LEGACY_SLUG_MAP`; 301 redirects + map removal moved to C6. Regression sweep + staging smoke green on v1.4.0-alpha-5.11.
A2 — Match-based play + match-level ELO	shipped	11/11	Historical rows frozen at cutover; new matches use the new formula. Corpus alignment audit landed an exact worked-example test pinned to `match-formats.md`.
A3a — Training data + UX rework (in-process)	shipped	12/12	All backend + corpus + tests done. Three UI items (stacked W/D/L viz, Master-vs-bot split, "no knobs" v1 disclosure) moved to A3b where they ship alongside the worker cutover.

Sprint	Status	Items	Notes
A3b — Training worker process cutover + training UI	complete	11/11	Sequenced as 11 PR-sized chunks. **Shipped:** A3b.1 spike (BullMQ + xo-training worker round-trip + Dockerfile lockfile fix); A3b.2a (worker can run the real training loop via `training:start` jobs in shadow mode — `_runTrainingForQueueJob` + QA endpoint + 4 unit tests; backend `setImmediate` path is unchanged); A3b.2b (flag-gated cutover — `startTraining` enqueues when `ml.useWorker=true`; cross-process pause/cancel via new `signalBus.js`; orphan resumer skips worker-dispatched sessions; 11 new unit tests + local smoke proved end-to-end pause across processes); A3b.3 (BullMQ knobs are SystemConfig-driven — `ml.workerConcurrency` + `ml.workerJobsPerSecond` read at worker boot; cap-trip rejects regardless of dispatch flag; +8 unit tests); A3b.4 (graceful SIGTERM coordinates with the loop via signalBus; per-process activeSessions registry; `attempts:3` + 30s/60s/120s exponential backoff on `training:start`; new admin `GET /training/dead-letter` enriches failed jobs with their TrainingSession summary; +15 unit tests); A3b.5 (observability: `queueMetrics` + per-worker `process.resourceUsage()` sampler writing to Redis with TTL + `trainingHealthMonitor` evaluating edge-triggered `queueStalled` / `deadLetter` alerts every 60s + new admin `GET /health/training`; +20 unit tests); A3b.6 (infra: `xo-training-staging` Fly app deployed + `ml.useWorker=true`; 1-hour 10-user × mixed-algo soak shipped 144 sessions, 138 COMPLETED in DB, 0 FAILED in DB, 0 DL

Sprint	Status	Items	Notes
A4 — Multi-skill bot UI + auto-clone	complete	7/7	All seven plan items shipped in one sprint. **Skills section on BotProfilePage** lists every `BotSkill` row with per-game ELO + last-trained timestamps and a **Primary** badge on the one tracking `User.botModelId` (data sourced from a now-enriched `GET /bots/:id` with `lastTrainedAt` derived from the newest COMPLETED `TrainingSession`). **Add-skill flow** opens `AddSkillModal` from the section header (owner-only); the modal gained an optional “Clone hyperparams from” dropdown that posts `cloneFromSkillId` to the existing `POST /bots/:id/skills`, which copies the source skill’s `config` blob verbatim (bogus id falls back to defaults rather than 404). **Rename preserves skills** — verified via a `PATCH /bots/:id` test that asserts no `botSkill.delete` / `deleteMany` ever fires; skills are FK’d by `botId` and carry through any `User` column mutation. **Picker disambiguation**: `BotCard` accepts a `pickerGameId` prop and appends `" (Game Label)"` to the displayed name when the bot has skills in ≥2 games; single-game bots stay un-suffixed; `TablesPage` wires `filters.gameId ?? gameId` through to the select-variant picker. **Profile → Gym deep-link**: each per-skill card renders a `Train in Gym →` link to /gym?bot=<id>&gameId=<gameId>, which `GymPage` already auto-resolves to `(botId, skillId)` via the existing `?bot=&gameId=` handler. **Pre-C4 validation** acknowledged — with only TTT live, the UI surface is lightly populated by design; Phase B adds the second game and the breadth lights

Sprint	Status	Items	Notes
Phase B — Connect 4 build			
B1 — `packages/game-connect-four/` package	blocked on A1+A3	0/7	Engine, Master solver, no UI.
B2 — C4 play surface (desktop + mobile)	blocked on B1	0/9	6×7 board, mobile column-input (tap-to-preview, tap-to-confirm).
B3 — C4 ranked + tournament	blocked on A2+B2	0/5	Reuses match infra from A2; no new ELO code.
B4 — C4 training, all five algorithms	blocked on A3+B1	0/8	Rule-Based / Minimax / MCTS / DQN / AlphaZero.
B5 — Master tier + Master Challenge	blocked on B1+B2	0/11	Perfect-play solver, off-ladder, best-of-2 Challenge format.
Phase C — Polish			
C1 — Journey + coaching updates	not started	0/3	New onboarding milestone post-first-ranked-TTT-win.
C2 — Training UX completions	not started	0/3	Stop-early, 7-day rollback, advanced disclosure.
C3 — Corpus completions	not started	0/3	Cost/preset docs queue + C4-specific examples.
C4 — Accessibility + mobile polish	not started	0/3	Keyboard nav, screen-reader labels, gesture conflicts.
C5 — Observability sweep	not started	0/2	C4 dashboards + alert thresholds.
C6 — Legacy slug cleanup	blocked on B exit	0/3	Drop `LEGACY_SLUG_MAP`, add 301 redirects from `/xo*`, scrub residual `xo` literals.
Total		**21/114**	

Update this table as sprints land. Each `- [ ]` flipped to `- [x]` in the body should be reflected in the `Items` column.

Goal

Ship Connect 4 as the platform's second game, alongside Tic-Tac-Toe, with:

- Strict SDK conformance (game logic lives entirely in `packages/game-connect-four/`; if the SDK is deficient, extend the SDK rather than smuggling game logic into the platform).
- Match-based play with alternating colors and match-level ELO (best-of-2 ranked, best-of-3 tournament, casual stays best-of-1).
- All five training algorithms (Rule-Based, Minimax, MCTS, DQN, AlphaZero) operating through a unified async training architecture with realtime W/D/L streaming, multi-curve eval, preset gating, checkpoint resume, and a dedicated worker process.
- A Master tier (perfect-play minimax solver, off-ladder) for the AlphaZero-teaching moment.
- Mobile column-input UI (tap-to-preview, tap-to-confirm).

Design principle: architecture before game

The first half of the plan touches **no Connect 4 code at all**. Every architectural change required by Connect 4 (slug rename, game-as-prefix routing, match-based play, training rework, worker process, multi-skill bot UI) is implemented and validated against the existing Tic-Tac-Toe game first.

Rationale: each architectural change is a potential regression vector. Doing them under the cover of "we're shipping a new game" tangles new-feature risk with refactor risk. Doing them on TTT first lets us regress-detect against a familiar surface, with no rollback ambiguity. Only after the platform reaches architectural parity does Connect 4 code begin.

This is enforced in the phase structure below: **Phase A is TTT-only**, **Phase B is the C4 build**, **Phase C is platform polish**.

Phase A — Architectural alignment (TTT only, no C4 code)

> **Five sprints.** Each ships through the normal `dev → staging → main` deploy flow. End state: TTT works identically to today from a user's perspective, but the platform underneath is ready to host a second game.

A1 — SDK audit + game-as-prefix routing + slug rename

Goal: Move TTT onto the routes and slug that the platform will use long-term, with the SDK contract extended to cover anything Connect 4 will need.

- ☒ SDK audit — confirm the current contract (per `Game_SDK_Developer_Guide.md`) suffices for a 6×7 column-input game with a perfect-play "Master" tier. Identify any gaps.
- ☒ Extend SDK if needed (likely: `meta.inputMode: 'cell' | 'column'`, `meta.matchFormat` hints, `meta.masterStrategy` hook for off-ladder perfect-play bots). Bump SDK version; update `Game_SDK_Developer_Guide.md`.
- ☒ Refactor `packages/game-xo/` to consume any new SDK fields explicitly. No hidden TTT assumptions.
- ☒ Add `/games/<slug>/...` route layer to backend (`/api/v1/games/<slug>/...`), landing (`/games/<slug>/...`), and tournament service.
- ☒ DB migration: rename `xo` → `tic-tac-toe` in `BotSkill.gameId`, `GameElo.gameId`, `Tournament.game`, and any other gameId-bearing rows. Single migration; no dual-write phase.
- ☒ Backend code: strip `GAME_ID = 'xo'` constant from `eloService.js`; parameterize every gameId-aware function.
- ☒ Backend cache keys: rename `bots:gameId:xo` → `bots:gameId:tic-tac-toe`. Flush on deploy.
- ☒ Landing: update `BotFilterBar.GAMES`, route definitions, deep-link parsers, picker UI.
- ☒ Update all docs in `/doc/Help_Corpus/` that reference the slug `xo` (search-and-replace pass).
- ☒ Tests: regression suite passes; e2e journey + smoke pass on staging.

> **Moved to C6** — 301 redirects from `/xo*` paths and `LEGACY_SLUG_MAP` removal. Doing them here would shorten the deprecation window to days; doing them in C6 (after Phase B ships) gives external callers — cached client bundles, bookmarks, anyone polling our API — multiple release cycles to migrate.

A2 — Match-based play + match-level ELO (TTT)

Goal: Ranked TTT becomes best-of-2 with alternating colors. Tournament becomes best-of-3 with random-color game-3 tiebreaker. ELO updates per match, not per game. Casual stays best-of-1.

- ☒ New `Match` entity in DB (or extend `Game` with `matchId`, `matchSequence`). Decide schema: pure relational `Match` row with child `Game` rows, or denormalized `matchId` column on `Game`.
- ☒ Match lifecycle states: `FORMING → IN_PROGRESS → COMPLETED`, with per-game results aggregated to a match score.
- ☒ Color assignment logic — game 1 random/seeded, subsequent games swap, game 3 tournament tiebreaker random with announcement.
- ☒ Ranked play: server orchestrates a best-of-2 match for any "Ranked vs X" entry point. No client-side color choice.
- ☒ Tournament play: extend existing `TournamentMatch` (`p1Wins`, `p2Wins`, `drawGames` already exist) to enforce best-of-3 with the random-color rule for game 3.
- ☒ `eloService.js`: implement match-score ELO update (sum of per-game 1.0/0.5/0.0 scores ÷ N games), replacing the per-game update for ranked/tournament. Casual single-game update unchanged.
- ☒ Historical rows: stay in place. No retroactive recompute. Pre-cutover ELO is the starting point; new matches move from there.
- ☒ Realtime: emit match-level events (`match.started`, `match.gameComplete`, `match.completed`) on the existing realtime channel. Client renders match progress (1-0, 1-1, etc.).
- ☒ Landing UI: ranked entry points say "Best of 2" explicitly. Match progress visible during play.
- ☒ Tests: ELO math worked-example test (matches the example in `Help_Corpus/match-formats.md`). E2E ranked-match flow. Tournament best-of-3 with 1-1 tiebreaker. *(Tournament BO3 e2e remains to add when tournament ELO moves to per-match — see corpus footnote.)*

- ☒ Corpus alignment: `matches-and-games.md`, `match-formats.md`, `match-design-rationale.md` are already published — confirm they describe the shipped behavior. *(A2.8 audit: fixed the "smaller weight" casual claim, replaced approximate worked-example numbers with the exact `+1.8 / +17.8 / -14.2` deltas the formula produces, footnoted that tournament BO3 still updates ELO per-game today.)*

A3a — Training data + UX rework (in-process, TTT)

Goal: Reshape the training API into the form Connect 4 will need, without yet moving training out of the backend process. TTT validates everything.

- ☒ **Tournament series-winner attribution: fix mark-vs-participant aggregation.** Surfaced during the A3a.1 audit. `socketHandler.recordPvpGame`'s tournament branch was persisting `TournamentMatch.p1Wins = xWins` and `p2Wins = oWins` and computing the series winner as `xWins >= oWins ? 'X' : 'O'` — both wrong whenever colors swap, which `rematchGame`'s tournament branch always does (game 1: seat1=X, game 2: seat1=O, game 3: random). Scenarios like *seat1 wins game 1 as X, seat1 wins game 2 as O* were getting stored as `xWins=1, oWins=1` (a "tie") when seat1 had actually swept 2-0. Fix: pulled the tournament branch's aggregation up to read `Game.winnerId` per `tournamentMatchId` (participant-keyed) and resolve host/guest → `TournamentMatch.participant1Id` / `participant2Id` by querying the row; both the series-done complete call and the mid-series persist now write participant-keyed totals. Tied series default to host (preserves the pre-A3a.1.5 "tied → X" fallback now that we no longer assume X=host across the whole series). Tests: 8 total — flag-on/off ELO, draw handling, HvB exclusion, 2-0 sweep with color-swap mismatch (the headline bug), inverted host/participant orientation, mid-series persist, all-draw BO3 fallback to host.
- ☒ **A2 carry-over: tournament match-level ELO (opt-in per game).** Audit during A3a kickoff turned up two facts the previous footnote got wrong: (a) tournament matches never updated ELO at all (the casual PvP ELO call has been guarded by `!isTournamentRoom` since tournament play first landed in 3843f49 — *"ELO update: skip for tournament games (design requirement)"*), and (b) the X/O mark-aggregated `xWins/oWins` the tournament branch carries aren't participant-relative because colors swap game-to-game. The fix is intentionally opt-in: a `tournamentMovesElo` flag on each game's SDK `meta`, mirrored into a `TOURNAMENT_ELO_GAME_IDS` set on the backend (`backend/src/constants/games.js`). TTT opts out (BO3 game-3 is a coinflip on a solved game); Connect 4 will opt in at launch. When the flag is on, `socketHandler.recordPvpGame` reads Game rows by `tournamentMatchId` and aggregates `winnerId` (participant-keyed) into a single `updateBothElosAfterMatch` call. Unit tests cover both flag states + draw handling + HvB exclusion. Corpus footnote in `match-formats.md` rewritten to describe the actual rule.
- ☒ **TrainingSession audit + TrainingCheckpoint + TrainingMetric.** Existing `TrainingSession` covered modelId/mode/iterations/status/config/summary/timestamps + `TrainingEpisode` children, but was missing presets, ETA gating fields, approval audit trail, pause/resume timestamps, and any checkpoint pointer. Additive migration (`20260519020000_a3a_training_session_audit`) added: `preset`, `expectedDurationMs`, `approvalStatus`, `approvalRequestedAt`, `approvedAt`, `approvedById` (FK → User), `pausedAt`, `checkpointEpisode` — all nullable so existing rows continue to work. New `training_checkpoints` table (sessionId, episodeNum, weights, runtimeState; unique on `(sessionId, episodeNum)`) supports checkpoint-based recovery. New `training_metrics` table (sessionId, episodeNum, opponentLabel, wins/draws/losses, `asFirstMover?`; indexed on `(sessionId, opponentLabel, episodeNum)`) supports stacked W/D/L curves + the deferred Master split. 6 schema tests cover field write/read, unique-checkpoint enforcement, multi-curve metric writes, asFirstMover, and FK cascade.
- ☒ **Channel rename: `ml:session:{id}` → `training:<sessionId>`.** One-for-one rename across 7 files (backend services + routes + landing UI + tests): channel prefix `ml:session:` → `training:` and each event type `ml:progress` / `ml:complete` / `ml:cancelled` / `ml:error` / `ml:curriculum_advance` / `ml:early_stop` rebadged to `training:*`. The existing "doubled-up" topic shape (`training:abc:training:progress`) is preserved — collapsing that to `training:abc:progress` would touch more files and isn't required for the SDK contract. Backend 2129/2129, landing 507/507 still green after the cut.
- ☒ **Preset model: Quick / Standard / Deep per (game, algorithm).** Catalog lives in `backend/src/config/trainingPresets.js`: one entry per (gameId × algorithm × preset) returning `{ iterations, expectedDurationMs }`. TTT covers all 7 trainable algorithms (qlearning, sarsa, montecarlo, policygradient, dqn, alphazero, mcts) + rule_based (zeroed presets, labelling-only). `resolvePreset()` normalises algorithm casing (Q_LEARNING / q-learning / qlearning all resolve to the same entry). Wired into `mlService.startTraining` + `startFrontendSession` and the equivalent skillService functions: preset overrides the iterations arg, persists `preset` + `expectedDurationMs` on the session row, throws "Unknown preset" for bad combinations (route returns 400). Catalog deliberately puts AlphaZero standard/deep + DQN deep + policygradient deep past the 4-hour mark so A3a.5 gating gets exercised on every trainable algorithm. Per-session iteration cap is preserved — "deep" qlearning/sarsa/montecarlo land at 100k, the hard cap. Tests: 12 catalog/resolver tests + 7 startTraining wiring tests.
- ☒ **ETA-threshold gating (backend).** In `mlService.startTraining` / `startFrontendSession`, after preset resolution: if

`expectedDurationMs >= ml.approvalThresholdMs` (default 4hr, configurable via SystemConfig), the session is created with `status='PENDING'` + `approvalStatus='NEEDED'` and the training loop is *not* started. Sub-threshold sessions auto-confirm and run as before. Legacy iterations-direct callers (no `expectedDurationMs`) are never gated. New service functions `approveSession(sessionId, adminUserId)` and `denySession(sessionId, adminUserId, reason?)` enforce state transitions (must be `PENDING + NEEDED`) and start the loop on approve (queuing if model is busy). Admin endpoints: `GET /admin/training/pending`, `POST /admin/training/:id/approve`, `POST /admin/training/:id/deny`. Tests: 10 new (gate trips for AlphaZero standard, skips for AlphaZero quick, respects SystemConfig override, no-preset never gated, approve idle/busy paths, deny with/without reason, list filter). **Admin queue UI is deferred** to the broader UI sweep (kept out of this sprint per the "no knobs in v1 UI" constraint — backend surface is complete and admin can hit the endpoints directly until then).

- ☒ **Per-user concurrency cap.** `_enforceUserConcurrencyCap(ownerBaId)` in mlService runs after preset resolution + per-model checks, before both gated and ungated session create. Counts the user's `status IN (PENDING, RUNNING)` rows joined to `BotSkill.createdBy = ownerBaId`. Default cap = 1 via `ml.maxActiveSessionsPerUser`; admins with the `ADMIN` role get an override via `ml.maxActiveSessionsForAdmin` (only takes effect when set to a non-zero value, otherwise the default still applies). Skills with `createdBy=null` (admin-seeded) are skipped — no owner to attribute to. Same check wired into `startFrontendSession`. Tests: 9 (default cap trip, config override, admin override, non-admin doesn't inherit, no-owner skip, PENDING + RUNNING both count, frontend variant).
- ☒ **Multi-curve eval.** New `backend/src/lib/evalCurves.js` is a pure module: per-algorithm primary-opponent catalog (ML algorithms → `minimax:hard`; rule_based → `minimax:medium`), `playEvalGame` / `runEvalSeries` (alternates model's mark across games so first-mover advantage washes out), `runEvalBatch` (one record per [primary, easy, medium] curve), and `recordEvalMetrics` (createMany into TrainingMetric). Default budget = 12 primary + 4 easy + 4 medium = 20 games/eval. Wired into `_runTraining`: every `EVAL_GAP=1000` episodes the loop calls `runEvalBatch` with the engine's no-explore `chooseAction(board, false)` as the model move fn and minimax of the right tier as the opponent. Persist is fire-and-forget; a transient eval failure logs a warning but never aborts training. 17 module tests (curve resolution, alternating-mark policy, illegal-move forfeit, batch fan-out, budget zero-skip, persistence shape).
- ☒ **Checkpoint-based recovery.** Two-part change. **Persist:** alongside the existing per-model `MLCheckpoint` write (kept for backward compat), `_runTraining` now also upserts a per-session `TrainingCheckpoint` row with `{ weights, runtimeState: { epsilon } }` and updates `TrainingSession.checkpointEpisode` — both at the existing 1000-episode cadence; upsert means a re-issued checkpoint after resume on the same slot is a no-op. **Resume:** new `resumeOrphanedSessions()` runs once at backend boot (`index.js`, fire-and-forget so it never blocks `listen()`). It finds every `status='RUNNING'` row, loads the latest `TrainingCheckpoint`, and either restarts `_runTraining` with `startEpisode` + `resumedEngineState` (weights + epsilon) — or marks the session `FAILED` and flips `BotSkill.status` back to `IDLE` if no checkpoint exists. `_runTraining` now accepts the resume params: weights replace the model's stored weights, and epsilon overrides the engine's initial value so exploration continues smoothly. Tests: 5 (no-orphans no-op, resume with checkpoint, FAILED no checkpoint, mixed batch, per-session error isolation).
- ☒ **Pause / Resume (backend).** Mirrors the cooperative-cancel pattern. `pauseSession(sessionId)` adds the session to an in-memory `pausedSessions` Set and returns the row; the training loop's next tick force-writes a TrainingCheckpoint at the current episode (so resume picks up exactly there, no replay), flushes the in-flight episode batch, transitions the session to `PENDING + pausedAt`, flips `BotSkill.status` back to IDLE, emits a `training:paused` event, and exits the loop without `_finishSession` so the row stays terminal-free. `resumeSession(sessionId)` is symmetric: rejects if not paused / no checkpoint, clears `pausedAt`, flips back to RUNNING, locks the model, and restarts `_runTraining` with the latest checkpoint's `startEpisode` + weights + epsilon (or queues if model is busy). `resumeOrphanedSessions` updated to skip `pausedAt`-set rows so a boot-time scan doesn't undo a user's explicit pause. Endpoints: `POST /api/v1/skills/sessions/:id/pause` + `.../resume` (owner-only via the existing `assertModelOwner` helper). Stop-early stays a Phase C concern. UI button deferred. Tests: 9 (pause requires RUNNING + unpaused, resume requires paused + checkpoint, queue when model busy, unknown session, all error paths).
- ☒ **Tests: TrainingSession + TrainingMetric CRUD, preset selection, ETA gating boundary at 4hr, multi-curve eval budget math, checkpoint resume after simulated crash.** Shipped piecewise alongside each item: 6 schema tests (CRUD + checkpoint uniqueness + metric writes + asFirstMover + FK cascade), 12 preset catalog + 7 startTraining wiring, 10 ETA-gate (incl. AlphaZero standard trips, quick skips, SystemConfig override, no-preset never gated, approve/deny paths), 9 per-user cap, 17 multi-curve eval (curve resolution, alternating mark, illegal-move forfeit, batch fan-out, budget zero-skip, persistence shape), 5 checkpoint resume (no-orphans no-op, resume with checkpoint, FAILED no checkpoint, mixed batch, per-session error isolation), 9 pause/resume, plus the `um training-recovery` harness (10 unit tests on `evaluateVerifyState`) and the manual 3-session parallel crash-recovery test now run end-to-end against staging.
- ☒ **Corpus.** Four new Help_Corpus docs: **`training-presets-and-time`** (Quick/Standard/Deep, the 4-hour admin gate, per-user concurrency cap, picking the right preset), **`training-multi-curve-eval`** (Primary/Easy/Medium reading, saturation fade, what each curve answers, ~20 games/eval budget), **`pausing-and-resuming-training`** (cooperative pause,

crash recovery, pause vs cancel decision), **`self-play-vs-eval`** (three kinds of games — training episode / eval game / live ranked — and how ELO is updated). Cross-linked back to the existing `understanding-training-charts` + `bot-training-concepts` + `bot-training-troubleshooting` so the new pages don't repeat material that's already published.

A3b — Training worker process cutover + training UI (TTT)

Goal: Move training out of the backend process into a dedicated worker, so a long AlphaZero Deep run can never starve the API box. Sequenced as 11 PR-sized chunks; each ships independently to dev and each de-risks the next. The three deferred A3a UI items (stacked W/D/L viz, Master-vs-bot split, "no knobs" disclosure) ride the same sprint because they consume data the worker now produces.

> **Runtime selector (A3b.10):** "Where does the loop run" is a routing decision per `(game, algorithm)`, not a global switch. TTT Q-learning is a few seconds in the browser and should stay there; AlphaZero Deep on C4 is hours of CPU and belongs on the worker. The A3b.2b `ml.useWorker` flag is the v0 — a single global. A3b.10 replaces it with a `(gameId, algorithm) → 'frontend' | 'backend-in-process' | 'worker'` matrix so we can preserve fast in-browser training where it makes sense and route the heavy stuff off-box where it has to go.

PR-sized chunks:

- ☒ **A3b.1 — Spike: new Fly app + BullMQ + hello-world job.** Stands up the queue plumbing without touching any real training behavior; proves Redis reachability + queue lib choice + a separate worker process. Files: `backend/src/queue/trainingQueue.js` (producer-side Queue + `enqueuePing`), `backend/src/queue/worker.js` (BullMQ Worker entrypoint, dispatch by job name, SIGTERM/SIGINT graceful shutdown), `backend/src/queue/jobs/ping.js` (pure handler, 3 unit tests). New QA_SECRET-gated `POST /api/v1/admin/qa/training-worker/ping` endpoint mirrors the training-recovery harness pattern so the round-trip can be smoke-tested against any env without a logged-in admin. New `backend/fly.training.toml` for the `xo-training-staging` / `xo-training-prod` apps — shares the backend image, entrypoint switched via `[processes] worker`. New `docker-compose.yml` `training-worker` service. Sidecar fix: both `backend/Dockerfile` and `backend/Dockerfile.dev` now copy `package-lock.json` and drop `--no-package-lock` — without the lockfile the workspace install was silently dropping bullmq transitives (tslib / msgpackr / node-abort-controller / cron-parser). Local verification: `POST /qa/training-worker/ping` → BullMQ → worker → handler → completed at ~7 ms enqueue-to-consume lag.
- ☒ **A3b.2a — Worker can consume `training:start` end-to-end (shadow mode).** Worker process boots, listens to a `training:start` job on the queue, and runs the existing `_runTraining` against the `qa-recovery-seed` skill (same fixture the crash-recovery harness uses). Backend continues to run the loop in-process for real traffic — this PR only proves the worker side. Files: new `backend/src/queue/jobs/trainingStart.js` pure handler (runner injected, 4 unit tests: happy path, missing-enqueuedAt, missing-sessionId throws, runner errors propagate); new `enqueueTrainingStart(sessionId)` in `trainingQueue.js`; worker.js `HANDLERS` map adds `'training:start'`. New `_runTrainingForQueueJob(sessionId)` export in `mlService.js` resolves session + model + latest checkpoint and awaits `_runTraining` — so a re-enqueued job resumes from the checkpoint episode (matches the orphan-resume path's semantics). New QA_SECRET-gated `POST /api/v1/admin/qa/training-worker/start` mints the session row, flips BotSkill to TRAINING, and enqueues. Local verification: 200-episode session enqueued → worker logged `training:start picked up` → handler logged `training:start completed` → session row COMPLETED with summary {159W/27D/14L, finalEpsilon 0.05}, BotSkill back to IDLE. Ping path unregressed.
- ☒ **A3b.2b — Cut over: backend enqueues instead of `setImmediate`.** Flag-gated by new `ml.useWorker` SystemConfig key (default false → unchanged in-process behavior). When true, `mlService.startTraining` mints the session tagged `config.dispatch:'worker'` and calls `enqueueTrainingStart(session.id)` instead of `setImmediate(_runTraining)`. Cross-process pause/cancel: new `backend/src/lib/signalBus.js` with two modes — `memory` (today's bare Sets, used when the flag is off) and `redis` (pub-sub: every process is a subscriber and mirrors signals into a local Set). Backend boots into the right mode based on the flag; the worker is always a subscriber. `pauseSession/resumeSession/cancelSession` write through signalBus instead of bare Sets; `_runTraining` reads via `_busIsPaused/_busIsCancelled`. `resumeOrphanedSessions` filters out `dispatch:'worker'` sessions (BullMQ + the stalled-job heartbeat own their recovery, not the boot scan). New QA_SECRET-gated `POST /qa/training-worker/{pause,cancel}` endpoints route through the long-running backend process so the publish actually flushes (the harness can't pause via a one-shot CLI subprocess because that process's signalBus is never initialized). Tests: 6 signalBus unit tests + 5 dispatch routing tests (flag-off → setImmediate + dispatch:in-process; flag-on → enqueue + dispatch:worker + no setImmediate; orphan scan skips worker-tagged sessions; legacy untagged orphans still resume; mixed orphans isolate correctly). Local smoke: real `startTraining` with flag on minted session tagged worker, BullMQ delivered to xo-training, training loop ran in worker, pause via HTTP through backend force-wrote checkpoint at ep 2132, session → PENDING, BotSkill → IDLE. SSE fan-out is incidentally free — the existing `events:tier2:stream`

(`backend/src/lib/eventStream.js`) is already Redis-backed, so worker-emitted progress events land in the same stream the backend's SSE replay reads. The 3-session parallel crash-recovery harness against the worker path is deferred to A3b.6 (staging soak); local pause-across-processes is the v0 acceptance.

- ☒ **A3b.3 — Cap enforcement + rate limits at the queue.** Two SystemConfig knobs (`ml.workerConcurrency` default 2, `ml.workerJobsPerSecond` default 4) feed BullMQ `Worker.concurrency` + `Worker.limiter` at worker boot — admins can tune throughput without a redeploy. New `backend/src/queue/workerOptions.js` factors the resolution logic into a pure function with injectable SystemConfig getter; worker.js logs the resolved values on boot so the live config is observable. Per-user cap (`_enforceUserConcurrencyCap`) already runs inside `startTraining` *before* the dispatch decision, so it gates both `setImmediate` and queue paths for free — A3b.3 adds explicit tests to pin that behavior across both flag states. Admin override via `ml.maxActiveSessionsForAdmin` is unchanged. The BullMQ OSS distribution doesn't offer per-`groupKey` rate limiting (that lives in the paid Pro variant), so the v0 sticks with a global rate limit; per-user fairness in the worker handler is deferred until a Pro-only feature actually justifies the upgrade. Tests: 6 workerOptions unit tests (defaults, configured, stringified, garbage→fallback, zero/negative clamp, both keys consulted) + 2 dispatch cap-trip tests (flag-off + flag-on, no setImmediate / no enqueue). Local smoke: set knobs to (3, 8), restart worker, boot log line confirmed; reset to defaults.
- ☒ **A3b.4 — Graceful shutdown + retry / dead-letter.** Worker SIGTERM handler walks the per-process active-session registry, raises the signalBus pause flag for each in-flight session, then awaits `worker.close()` so the training loop's next tick force-writes a checkpoint and the row transitions to PENDING + pausedAt before the process exits. New `backend/src/queue/activeSessions.js` (Set-based registry; `handleTrainingStart` wraps the runner in try/finally so a thrown runner still unregisters and the shutdown path never waits on a zombie entry). `enqueueTrainingStart` adds `attempts:3` + exponential backoff (`{ type:'exponential', delay:30_000 }` → 30s, 60s, 120s); `removeOnFail` left default so exhausted jobs stay in BullMQ's `failed` index. New admin `GET /api/v1/admin/training/dead-letter` reads the failed set (configurable `?limit=`, hard ceiling 200), enriches each `training:start` job with its TrainingSession row (status, summary, model, checkpoint), truncates `stacktrace` to first 3 lines for the list view. Tests: 6 activeSessions registry tests + 2 handleTrainingStart lifecycle tests (register/unregister around success, register/unregister around throw) + 3 enqueueTrainingStart job-options tests + 4 DL-endpoint tests = 15 new. The live SIGTERM-mid-flight wall-clock test is deferred to A3b.6 (stage soak) because the local background-task scaffolding makes the timing window unreliable — the contract pieces are independently covered above.
- ☒ **A3b.5 — Observability.** Three new modules wire training-worker health into the admin surface. `backend/src/queue/queueMetrics.js` reads BullMQ `getJobCounts()` + derives `oldestWaitingAgeMs` from the head of the waiting list (with timestamp-vs-data fallback and a defensive null-on-race for `getWaiting`). `backend/src/queue/workerResourceSampler.js` runs *inside the worker process* — it samples `process.resourceUsage()` + `process.memoryUsage()` every 30s, computes CPU% deltas vs the previous sample (clamped to 0 against counter regressions), and writes to Redis under `training:worker:metrics:<workerId>` with a 120s TTL (so a dead worker auto-evicts within one missed heartbeat); a second IORedis client is used because BullMQ reserves its own `connection`. `backend/src/queue/trainingHealthMonitor.js` runs in the backend process — every 60s it pulls queue metrics + reads each worker's sample (via `redis.keys + mget`) and evaluates two edge-triggered alerts: `queueStalled` (waiting>=N **and** oldestWaitingAgeMs>=5min — both conditions must hold so a fresh burst doesn't fire), and `deadLetter` (failed>=1). Alerts dispatch via the existing `notificationBus` to admins; cleared transitions also dispatch so a noisy on/off pair doesn't get lost. New `GET /api/v1/admin/health/training` returns the latest snapshot + current alert flags + uptime — the admin Health page can render the gauge inline (Bull-Board deferred). Worker SIGTERM stops the sampler + del's its Redis key before exit. Tests: 5 queueMetrics + 5 workerResourceSampler + 7 trainingHealthMonitor + 3 admin-endpoint = 20 new (all green; full backend suite 2271/2271). `Observability_Plan.md` extended with a "Training worker queue" section pointing at the new keys + thresholds.
- ☒ **A3b.6 — Stage soak + promote (soak passed; ready for `/promote`).** Infra: `xo-training-staging` Fly app created + deployed from `backend/fly.training.toml` (2× shared-cpu/2/1gb in iad, secrets mirrored from staging backend). `ml.useWorker=true` set in staging SystemConfig; staging backend restarted so signalBus subscribes to Redis and `trainingHealthMonitor` boots; worker boot log confirmed `concurrency=2 jobsPerSecond=4 training worker ready`. QA endpoint extended: `POST /qa/training-worker/start` accepts `algorithm` body param + `_qaEnsureSeedSkill(slot, algorithm)` mints per-(algorithm, slot) skill rows. **Soak run** via new `backend/src/scripts/trainingWorkerSoak.js` (10 virtual users × mixed algos `qlearning:6,sarsa:2,monte_carlo:2`, 5000 iters × 10 ms delay per episode, 61 min wall-clock): **144 sessions** started, **138 COMPLETED in DB**, **0 sessions FAILED in DB**, **0 dead-letter entries**, **no `queueStalled/deadLetter` alerts fired**, worker process never restarted (single pid the whole run, 174 MB RSS steady, ~6 % CPU). 8 sessions the harness initially logged as "FAILED" were transient: A3b.4's `attempts:3` + exponential backoff caught the error, BullMQ retried, and the worker resumed from the last checkpoint to COMPLETED — exactly the design intent. 6 trailing "TIMEOUT" rows are sessions still in-flight at the harness deadline; the post-soak queue drained to `waiting=0 failed=0`. The 3-session parallel crash-recovery scenario folds into this: every

"FAILED→retry→COMPLETED" sequence is the same recovery path the original A3b.6 acceptance called for, just driven by the retry policy instead of a SIGTERM. Observation gap to flag for A3b.5 follow-up: `redis.keys()` only returned one worker sample even though the standby machine exists — confirmed: standby is Fly HA, not a parallel-work consumer (one pid actively running). **Next:** you invoke `/promote` to push v1.4.0-alpha-5.16 → main; prod gets the worker app + flag via the same `fly deploy --config backend/fly.training.toml --app xo-training-prod` recipe + matching SystemConfig flip.

- ☒ **A3b.7 — Stacked W/D/L visualization (moved from A3a).** New `GET /api/v1/ml/sessions/:id/metrics` endpoint surfaces `TrainingMetric` rows the A3a.7 multi-curve eval already writes (ordered `(episodeNum asc, opponentLabel asc)` so the chart can render incrementally). New `landing/src/components/gym/StackedCurvesChart.jsx` component groups by `opponentLabel` and renders one recharts `AreaChart` per curve (`primary` / `easy` / `medium` / `master` in that order; unknown labels fall to the end). Each chart shows wins/draws/losses as a 100%-stacked area over `episodeNum` — normalizing to percent so curves with different eval budgets are visually comparable on the same y-axis. **Saturation fade**: per-curve detector wraps the chart at `opacity:0.4` + "(saturated — wins ≥ 100% for last 3 eval points)" badge when the last `SATURATION_WINDOW=3` eval points are all 100% wins (defensive against `total=0`). Wired into TrainTab in two places: live during training (polls every 5s so new eval points fill in), and in the post-session "Last Training Summary" card (no polling — eval points are immutable). Component renders nothing when the session has <2 eval points per curve (recharts can't draw a single-point area + saturation needs ≥3 points anyway) or when there are no multi-curve metrics at all (legacy single-opponent sessions). New helper `api.ml.getMetrics(sessionId)`. Tests: 4 endpoint + 5 `isCurveSaturated` unit + 4 component render = 13 new (backend 2293/2293 green; landing 520/520).
- ☒ **A3b.8 — "No knobs" disclosure (moved from A3a).** Default v1 training surface is presets-only — Mode + read-only Algorithm + a `Quick / Standard / Deep` picker + Train. New `GET /api/v1/ml/presets?gameId=&algorithm=` endpoint returns the existing `TRAINING_PRESETS` table; `api.ml.getPresets()` helper fetches it; picking a preset writes through to `iterations` state so the kickoff sends the right episode count whether the slider is rendered or not. New public `GET /api/v1/config/features` endpoint reads SystemConfig key `features.trainingAdvancedKnobs` (default false; fails-open to false on outage so accidental exposure isn't a failure mode). New `useFeatures()` hook on landing fetches once per mount. TrainTab consults `useOptimisticSession()` for role and `useFeatures()` for the flag: `showAdvanced = isAdmin || !!features.trainingAdvancedKnobs`. When false, the full knob set (VS_MINIMAX difficulty + Play-as, DQN config, AlphaZero config, epsilon decay, curriculum, custom iterations slider, early-stop) is gated out; the user sees only Mode → Algorithm → Preset picker → Train. Admins see an "Advanced" section header that surfaces the original controls inline. Tests: 4 presets endpoint + 4 `useFeatures` hook + 5 TrainTab disclosure (default-hidden / admin-visible / flagged-visible / preset-picker-rendered / preset-click-selects) = 13 new (backend 2297/2297 green; landing 529/529).
- ☒ **A3b.9 — Master-vs-bot split stub (moved from A3a; lights up in Phase B).** New pure `splitByFirstMover(points)` helper in `StackedCurvesChart.jsx` returns `{split:false}` when every row has `asFirstMover=null` (TTT default — no Master tier) or only one side is populated; returns `{split:true, firstMover, secondMover}` only when both sides are present. The chart's render loop refactored into a `renderCurve(...)` inner helper called once per single curve OR twice for a split curve (first-mover + second-mover sub-charts). Sub-charts get distinct test-ids stacked-curve-<label>-firstmover and `-secondmover` so per-half saturation/visibility can be asserted independently. The TTT default path is unchanged — `splitByFirstMover` short-circuits, the single stacked-curve-<label> test-id renders as before, and zero new HTTP traffic is added. Lights up automatically when Connect 4's Master solver writes split metrics (the `TrainingMetric.asFirstMover` column already exists from A3a.7). Defensive: rows with `asFirstMover=null` in a split curve land in BOTH halves rather than getting dropped. +7 unit tests (4 `splitByFirstMover` rule + 3 component render: unsplit-master-stays-single / split-master-renders-two / per-half-saturation); landing 536/536.
- ☒ **A3b.10 — Per-(game, algorithm) training-runtime selector.** Replaced A3b.2b's single global `ml.useWorker` flag with a routing matrix keyed on `(gameId, algorithm)` whose cells are `'frontend' | 'backend-in-process' | 'worker'`. New `backend/src/services/trainingRuntime.js` exports a pure `resolveTrainingRuntime(gameId, algorithm, { getConfig })` resolver that consults `ml.runtimeMatrix` SystemConfig first, then falls back to legacy `ml.useWorker` (so existing prod config keeps working until an admin sets the matrix), then to a frozen `DEFAULT_MATRIX` in code: TTT qlearning/sarsa/monte_carlo → `'frontend'`, everything else (DQN, AlphaZero, all C4 algorithms) → `'worker'`. Lookup precedence: `matrix[gameId][algorithm]` → `matrix[gameId].default` → `matrix.default` → legacy useWorker → code default. Garbage cells (admin typo'd "wroker") fall through to the next layer instead of crashing dispatch. `mlService.startTraining` now consults the resolver instead of the bool and tags the session row with `config.runtime`. New public `GET /api/v1/ml/runtime?gameId=&algorithm=` endpoint (Cache-Control: no-store so admin edits land immediately) lets the UI consult before kickoff. `landing/src/components/gym/TrainTab.jsx` calls `api.ml.getRuntime()` pre-kickoff, branches: `'frontend'` runs the existing in-browser loop; `'worker'` / `'backend-in-process'` POSTs `train` without `frontend:true`, sets `watchedSessionId` so the existing SSE subscription picks up progress events the backend/worker publishes to training:<sessionId>:progress, and shows a `data-testid="train-background-banner"` "Training in background — you can close this tab" affordance. `handleCancel` calls `api.ml.cancelSession` for backend

sessions instead of just flipping a local ref. Resolver lookup failures (Redis down, route 5xx) fall back to the frontend path so a backend hiccup never blocks training. Tests: 14 trainingRuntime unit + 4 admin `/ml/runtime` endpoint + 3 `api.ml.getRuntime` helper = 21 new (backend suite 2289/2289 green; landing 511/511).

A4 — Multi-skill bot UI + auto-clone

Goal: Finish the Phase 3.8 remainder. By the time Connect 4 ships, multi-game bot management is already in users' hands.

- ☒ **Bot detail page: per-skill cards.** New `Skills` section on `BotProfilePage` renders one card per `BotSkill` with game label, algorithm, ELO (rounded), and a "Last trained" date sourced from the newest `COMPLETED TrainingSession` per skill (Prisma `groupBy` on `modelId`). The card matching `User.botModelId` gets a `Primary` badge (test-id `bot-profile-skill-primary-<gameId>`). Section auto-hides on skill-less bots so brand-new identity bots aren't shown an empty box.
- ☒ **Add-skill flow + `cloneFromSkillId`.** Owner-only `+ Add skill` trigger in the section header opens the existing `AddSkillModal`; the modal gained an optional "Clone hyperparams from" dropdown that lists the bot's existing skills. `POST /bots/:id/skills` now accepts `cloneFromSkillId` and copies the source skill's `config` blob verbatim (validated to belong to the same bot — you can't lift another owner's config; unknown id falls back to `{}` rather than 404). First-skill bots still inherit the new skill as primary via the existing `botModelId`-when-null branch, so `repointBotPrimarySkill` (the training-finalisation path) keeps owning the rename-to-primary semantics it already does.
- ☒ **Rename carries skills.** Verified by a `PATCH /api/v1/bots/:id` test that asserts no `botSkill.delete` / `deleteMany` ever fires during a rename; skills are FK'd by `botId` and survive any `User`-column mutation. The handler updates only `User.displayName` — there's no skill-touching path to regress.
- ☒ **Picker disambiguation.** `BotCard` accepts a new `pickerGameId` prop; when the bot has ≥2 entries in `playableGameIds` AND the picker is scoped to one game, the displayed name becomes `"<Name> (<Game Label>)"`. Single-game bots stay un-suffixed (the parenthetical would be noise). `TablesPage` wires `filters.gameId ?? gameId` through so the create-table picker disambiguates naturally; the unknown-game fallback shows the raw `gameId` instead of crashing.
- ☒ **Profile → Gym deep-link.** Each skill card renders a `Train in Gym →` link to `/gym?bot=<id>&gameId=<gameId>`. `GymPage` already auto-resolves `?bot=&gameId=` to `(botId, skillId)` via the existing handler (lines 124/147), so the deep-link wires straight into the per-skill detail panel with no new routing.
- ☒ **Pre-C4 validation.** Acknowledged — with only TTT live the picker disambiguation, the add-skill game dropdown, and the Skills section are all intentionally lightly populated. Phase B's `connect-four GAMES` entry + per-bot Connect 4 skills will exercise the breadth automatically with no further UI work.
- ☒ **Tests.** 5 backend (lastTrainedAt enrichment on `GET /bots/:id`, `cloneFromSkillId` happy path, `cloneFromSkillId` bogus-id fallback, `cloneFromSkillId` 400 validation, rename-keeps-skills) + 4 `BotCard` picker-disambiguation (multi-skill + suffix / single-skill no suffix / no `pickerGameId` no suffix / unknown game fallback) + 6 `BotProfilePage.skills` (renders one card per skill / primary badge / ELO + lastTrained vs "Not trained yet" / owner sees Add + Train / non-owner sees neither / skill-less bot hides section). Total: **15 new tests**. Backend suite 2302/2302; landing 546/546.

Phase A exit criteria

- ☐ TTT plays identically end-to-end (casual + ranked + tournament) on the new routes and new slug.
- ☐ TTT match-level ELO produces sensible movements for the existing user base.
- ☐ All 5 TTT algorithms train through the worker process with W/D/L streaming, multi-curve eval, and checkpoint resume.
- ☐ Multi-skill bot UI lets a user add, manage, and queue from multiple skills (limited to one game today).
- ☐ Help corpus updated: any TTT-facing docs that reference the old slug, single-game ranked, or the old training UX are refreshed.
- ☐ No open regressions on the V1 acceptance script.

Phase B — Connect 4 implementation

Five sprints. Each builds atop the architecture established in Phase A. If any sprint here surfaces an SDK gap, the work pauses, the SDK is extended (with `Game_SDK_Developer_Guide.md` updated), and the sprint resumes — game logic never leaks into the platform.

B1 — `packages/game-connect-four/` package

Goal: A standalone, SDK-conformant game package. Engine + bot interface + tests, no UI yet.

- ☐ Package scaffold: `packages/game-connect-four/` with the same shape as `packages/game-xo/`.
- ☐ Engine: 6×7 board, gravity mechanic, win detection (horizontal/vertical/both diagonals), draw detection (board-full), legal-move enumeration (top-of-column).
- ☐ State serialization: compact format suitable for replay storage and bot weight inputs.
- ☐ `meta` export: `id: 'connect-four'`, `inputMode: 'column'`, `matchFormat: { ranked: 'bo2', tournament: 'bo3', master: 'bo2' }`, `masterStrategy: 'solver'`, `builtInBots: [easy, medium, hard, master]`.
- ☐ `botInterface`:
  - Built-in minimax tiers (Easy depth-2, Medium depth-4, Hard depth-6 to depth-8, Master = the solver).
  - Training hooks for the five algorithms (delegating to the shared `@xo-arena/ai` engines where game-agnostic; new code only where C4 specifics matter).
- ☐ Master solver: perfect-play minimax with full-depth search and the known solved-game heuristics (center-first opening, threat parity). Off-ladder.
- ☐ Unit tests: engine correctness (win/draw/illegal-move), serialization round-trip, minimax depth correctness, solver vs known opening lines.

B2 — C4 play surface (desktop + mobile)

Goal: Casual quick match against built-in Easy/Medium/Hard works end-to-end via `/games/connect-four/...`.

- ☐ Game component: 6×7 board with Red/Yellow tokens, drop animation, win highlight, draw indicator.
- ☐ Desktop input: click on column to drop.
- ☐ Mobile input: tap-to-preview (semi-transparent token at the top of the column showing where the piece will land), tap-to-confirm (drops the piece). Tap a different column to switch preview. Long-press cancels.
- ☐ Audio cues: drop, win, draw — reusing the existing `sdk.playSound()` pattern with new sample IDs.
- ☐ Theming: Red/Yellow color scheme conforms to platform tokens (`packages/sdk` theme exports).
- ☐ Spectate: rendered-table mode works; piece drops animate for spectators.
- ☐ Replay: replays of C4 games render via the same replay infrastructure as TTT.
- ☐ Bot vs human: server-side bot moves work for built-in Easy/Medium/Hard. Master is not yet wired here (Phase B5).
- ☐ Tests: e2e casual match vs each built-in tier on desktop + mobile viewports.

B3 — C4 ranked + tournament

Goal: Ranked best-of-2 and tournament best-of-3 work for C4 with zero new infrastructure — A2's match layer just lights up.

- ☐ Wire C4 into ranked match orchestration. Alternating colors, ELO updates per match.
- ☐ Wire C4 into the tournament service. Best-of-3 with random-color game 3 tiebreaker.
- ☐ Classification ladder: C4 ELO tracked separately from TTT (already supported by `GameElo (userId, gameId)` unique constraint).
- ☐ Landing nav: C4 appears in the games picker; BotFilterBar shows both games.
- ☐ Tests: ranked best-of-2 happy path, tournament with 1-1 game-3 tiebreaker, classification ladder shows independent TTT and C4 ratings for the same user.

B4 — C4 training, all five algorithms

Goal: Every algorithm trains through the A3 architecture with no platform-side changes.

- ☐ Rule-Based: heuristic rules (center preference, win-on-move, block-on-move, fork detection, edge play). Zero-time "training."
- ☐ Minimax (Quick Bots): tier-label bump only, same pattern as TTT Quick Bots — `user:<id>:minimax:<tier>` for C4.
- ☐ MCTS: classical UCB1 with rollouts. Tunes simulation count, exploration constant, optional rollout policy.
- ☐ DQN: experience replay + target network. Standard preset hyperparams.
- ☐ AlphaZero: policy/value network with PUCT-guided MCTS. Three presets:
  - Quick — ~30 minutes, small network, low simulation count. Auto-confirmed.
  - Standard — ~4 hours, medium network. **Hits admin-approval threshold.**
  - Deep — ~40 hours, full network + high sim count. Always admin-approved.
- ☐ Eval curves: primary opponent per algorithm (e.g., AZ evals against Hard minimax). Easy + Medium side curves render saturated and auto-fade.
- ☐ Streaming UX: TTT users seeing W/D/L curves in realtime today will see the same UI for C4 with no relearning.
- ☐ Tests: each algorithm reaches expected milestones from a known seed in CI-bounded episode counts.

B5 — Master tier + Master Challenge

Goal: Perfect-play Master, off-ladder, with the Master Challenge user flow.

- ☐ Master button on the C4 home: opens a best-of-2 Master Challenge (one game as first-mover, one as Master first-mover).
- ☐ Master plays the perfect-play solver from B1. Move latency budgeted (solver is fast enough at depth-required for solved-game optimality, but cache opening lines).
- ☐ No ELO update on Master matches. Per-bot "vs Master" stat block on the bot detail page (wins / draws / losses).
- ☐ Training integration: AlphaZero bots can opt into "vs Master" eval as an additional curve (showing first-mover-draw rate over time — the AlphaZero teaching moment).
- ☐ Landing UI: Master section on C4 home page explains the off-ladder rule and what "beating Master" means (linking to `Help_Corpus/solved-games-and-master.md`).
- ☐ Tests: Master Challenge happy path, off-ladder ELO behavior (no rating movement), AZ training with Master-vs-bot eval curve.

Phase B exit criteria

- ☐ C4 ships through `/games/connect-four/...` for casual, ranked, tournament, and Master Challenge.
- ☐ All five algorithms train through the A3/A3b architecture. AZ Deep on C4 succeeds at least once on staging without affecting API latency.
- ☐ Multi-skill bots can hold both a TTT and a C4 skill, each with independent ELO.
- ☐ Replay, spectate, ranked classification all work for C4 at parity with TTT.
- ☐ Corpus matches reality — the Connect 4 pages in `Help_Corpus/` reflect what shipped.

Phase C — Polish

Approximately 3-4 sprints. Lower urgency than Phase B but completes the end-to-end experience.

C1 — Journey + coaching updates

- ☐ Update the onboarding journey to surface a Connect 4 milestone after the user's first ranked TTT win.
- ☐ Add coaching cards for C4 specifics (column thinking, first-mover advantage, threat parity).
- ☐ Update `Intelligent_Guide_Requirements.md` and `Intelligent_Guide_Implementation_Plan.md`.

C2 — Training UX completions

- ☐ Stop-early button on active sessions (with confirmation + "save current checkpoint").
- ☐ 7-day rollback button on bot detail page: revert a skill's weights to a previous checkpoint within a 7-day retention window.
- ☐ Advanced disclosure (v1.1): expose hyperparameter knobs behind an "Advanced" toggle for users who want to tune past presets.

C3 — Corpus completions

- ☐ Write the queued cost/preset docs: "What each preset does", "How long does training take?", "What does training cost?", "What happens if I close the tab?", "Stop training early", "Why no knob tweaking", "Why AZ slow on C4 fast on TTT".
- ☐ Refresh `bots-overview.md`, `gym-train-tab.md`, `understanding-training-charts.md` with C4-specific examples.
- ☐ Render PDFs via the tuned pandoc invocation.

C4 — Accessibility + mobile polish

- ☐ Keyboard navigation for the C4 board (arrow keys + enter to drop).
- ☐ Screen-reader labels for board state ("Column 4, row 3, your turn").
- ☐ Mobile UX refinement pass — confirm the tap-to-preview-tap-to-confirm gesture works cleanly across viewports and doesn't conflict with browser swipe gestures.

C5 — Observability sweep

- ☐ C4-specific dashboards: completion rate per tier, AZ Master-draw-rate over time, training queue depth, worker CPU/RAM under typical load.
- ☐ Alert thresholds: dead-letter queue depth, training session failures per hour, API latency regression during heavy training.

C6 — Legacy slug cleanup

Goal: Drop the `xo` → `tic-tac-toe` alias once Phase B has shipped and external callers have had multiple release cycles to migrate. Cleanup, not architecture — sequenced here on purpose so the deprecation window is long enough that nothing in the wild still calls the old slug.

Pre-conditions: Phase B is shipped to production. No staging or production access logs show requests with `gameId=xo` or path `/play/xo*` for at least one release cycle (≥ 2 weeks). Verify via the request logs / observability dashboards before starting.

- ☐ Add `301 Moved Permanently` redirects in `landing/server.js` and (if needed) the backend for legacy paths: `/play/xo*` → `/games/tic-tac-toe/play*`, `/xo/*` → `/games/tic-tac-toe/*`. Keep the redirects in place for one further release.
- ☐ Remove `LEGACY_SLUG_MAP` from `backend/src/constants/games.js` and `tournament/src/constants/games.js`. `resolveGameSlug('xo')` should now return `null` (caller's `?? rawGameId` fallback then surfaces "Unknown game slug" 404 via `validateGameSlug`).
- ☐ Sweep for any residual `'xo'` literals in code (excluding migration files, historical perf-trace doc citations, and Help_Corpus search-alias tags). Update or delete each one.
- ☐ Update `Game_SDK_Developer_Guide.md` to drop the "legacy alias accepted" caveat — kebab-case canonical slugs only.

Risks + mitigations

Risk	Mitigation
Slug-rename migration corrupts production data. Renaming `xo` → `tic-tac-toe` across `BotSkill`, `GameElo`, `Tournament` is a write-heavy migration.	Dry-run on staging with a prod data snapshot. Migration runs inside a transaction with explicit row counts logged. Rollback plan: a reverse-rename migration is staged before deploy.
Match-level ELO produces unexpected user-visible drops or jumps at cutover. Even with frozen historical rows, the first few matches under the new formula will move ratings under different math.	Communicate proactively in release notes. The new math gives smaller per-match deltas (it's "match score 0.5" vs "game-loss 0.0"), so the practical effect is gentler movement. Monitor for outliers.
AlphaZero Deep run on C4 starves the API box despite the worker. Per-user cap and worker isolation should prevent this, but a misconfigured Fly app could fall back to in-process.	Pre-launch load test on staging: queue a real AZ Deep run while a separate user plays ranked. API p95 latency must not regress.
SDK gap surfaces mid-Phase B. Discovering during B2 that the SDK can't express column-input semantics would block the sprint.	Phase A1 audits explicitly for this. If a gap appears in B, the SDK is extended in place; we don't ship workarounds.
Master solver is too slow at depth-required. Perfect-play C4 needs deep search; if move latency is too high, the Master tier feels broken.	Cache the entire opening book (first 10-12 plies) as a lookup table. Worst case, fall back to a precomputed move table for the opening, switch to live search after the book exits.
Mobile column-input gesture conflicts with browser scroll. A column tap shouldn't scroll the page.	`touch-action: manipulation` on the board; e2e tests on mobile viewports.
Worker deployment introduces a new failure surface. A worker outage could halt all training.	Status page reflects worker health. If the worker is down, new training-start requests return a clear "training temporarily unavailable" error and queue UI surfaces the outage. Existing ranked/casual/tournament play is unaffected (those don't go through the worker).

Sequencing summary

```
Phase A (TTT only):
  A1 — SDK + routing + slug rename
  A2 — Match-based play + match-level ELO
  A3a — Training data + UX rework (in-process)
  A3b — Worker process cutover
  A4 — Multi-skill bot UI + auto-clone

Phase B (Connect 4 build):
  B1 — packages/game-connect-four/
  B2 — C4 play surface (desktop + mobile)
  B3 — C4 ranked + tournament
  B4 — C4 training, all five algorithms
  B5 — Master tier + Master Challenge

Phase C (polish):
  C1 — Journey + coaching
  C2 — Training UX completions
  C3 — Corpus completions
  C4 — Accessibility + mobile polish
  C5 — Observability sweep
```

Phases ship sequentially. Sprints within a phase can run in parallel where they don't have ordering dependencies (e.g., B2 and B3 can overlap; B4 needs the C4 package from B1; B5 needs the C4 surface from B2).

Open items (will be answered as the plan executes)

- **Worker pool sizing.** Start with `count=1` on staging, `count=2` on prod, scale based on observed queue depth. Final sizing TBD post-launch.
- **AZ C4 Deep economics.** Per the corpus, AZ C4 Deep is roughly $3–50 per run depending on Fly machine class. The exact admin-approval UX (credits charged? burned against an admin pool?) is open — pin down before Phase B4.
- **Master solver move-cache size.** Opening-book lookup table size depends on how deep the book runs. Spike during B1.
- **Color naming.** Red/Yellow is canonical; settle whether colorblind-mode swaps to a high-contrast pair or uses pattern fills. Decide during B2.